MZ2POL-09: Migrating Management Zone-Scoped Alerting and Notifications

> **Series:** MZ2POL — Management Zone to Policy Migration | **Notebook:** 10 of 10 | **Created:** July 2026 | **Last Updated:** 07/31/2026

Overview

Management Zones do three jobs. Notebooks 02–04 and 06–08 cover the **access-control** job; notebook 05 covers the **filtering** job. This notebook covers the third: **alerting**.

An alerting profile scoped to a Management Zone decides who gets paged. When the zone goes away, that scoping has to land somewhere — and it does not land on Segments. This notebook covers where it actually lands, what it costs, and the one step of the migration whose failure mode is undocumented.

> ⚠️ **If you read one thing here, read §2.** The claim that alerting profiles will be "bound to Segments" circulated widely — including in earlier revisions of this series — and it is wrong. Teams that believe it *defer* the alerting work, waiting for a feature that is not coming. That deferral is what turns a manageable migration into a cutover crisis, because the alerting work is the long pole, not the short one.

Table of Contents

Prerequisites

Requirement	Details
Environment	Dynatrace Gen3 SaaS with Grail
Permissions	Workflow creation; Settings read for alerting-profile export; IAM policy authoring for §7
Non-prod tenant	**Required** for §8 — the deletion test must not be run first in production
Prior reading	**MZ2POL-01** §5 (the MZ-job-to-successor mapping) · **MZ2POL-03** (assessment) · **MZ2POL-05** (the filtering job, for contrast)
Companion series	**WFLOW-02** (trigger configuration) · **WFLOW-04** (routing patterns) · **ALERT-01** (alerting architecture) · **ALERT-03** (destinations and cost)
Tagging prerequisite	The routing dimensions must exist as **entity tags** before any workflow can filter on them — see §4

1. The Third Job

A Management Zone conflates three concerns that are three separate mechanisms on Gen3.

Axis	Question it answers	Gen3 mechanism	Where it is covered
Filtering	What does a user *see* in the UI?	Segments	MZ2POL-05
Access	What data may a user *read*?	Policies + boundaries on `dt.security_context`	MZ2POL-02/04
Routing	Who gets *paged*?	**Problem-triggered workflows**	**This notebook**
Visibility	Who may *see the problem* at all?	IAM record-level policy on `dt.security_context`	§7 below

Routing and visibility are related but distinct, and §7 explains why conflating them produces a model that silently does not work.

Why the alerting job is the hard one

Filtering and access both migrate to constructs that can evaluate a **condition**. A segment can filter on `contains(entity.name, "payment")`; a boundary can test a `dt.security_context` prefix.

A workflow trigger cannot. It matches **tags carried by the affected entity**, plus an optional DQL matcher on the problem record. That is a narrower surface, and it means a Management Zone whose rules are *computed* — name patterns, entity selectors — has nothing for a trigger to filter on until equivalent tags exist.

That asymmetry is the single biggest source of surprise in this part of the migration, and §4 is devoted to it.

2. What Actually Replaces an Alerting Profile

It is not a segment

There is **no segment field on an alerting profile, and no segment field on a workflow trigger.** Segments are query-time filter conditions — they scope what a *query* returns and what a *user sees*. They do not scope what *fires*.

Dynatrace's alert-notification upgrade guide is explicit: the **Management Zone filter is "no longer supported. Use Grail record-based field filters instead."** There is no successor filter inside the alerting model.

Segments touch alerting in exactly one documented place: **scoping an anomaly detector** (Anomaly Detection app → *Set scope* → *Segments*). That scopes **detection** — which signals are evaluated — not **routing**.

> **Where the misconception came from.** The claim traces to a Dynatrace community forum answer rather than to documentation, and it propagated into this series: MZ2POL-01, -03, -05, -06, and -99 all carried some version of "alerting profiles → Segments (upcoming)" until July 2026. Independent corroboration that it was never the direction: Dynatrace's current alerting and notifications hub links to problem triggers, Davis event triggers, the Anomaly Detection app, and workflow connectors — and does not link to alerting profiles at all.

It is a problem-triggered workflow

An alerting profile scoped to MZ *X*, routing to team *X*'s channel, becomes:

```
Workflow  "Route — Team X"
  Trigger:  Problem trigger
            ├─ Problem state ......... Active
            ├─ Severity .............. (as per the old profile's severity
rules)
            ├─ Affected entities ..... include entities with tag  team:X
            └─ Custom filter query ... optional DQL matcher, e.g.
                                       maintenance.is_under_maintenance ==
false
  Action:   Slack / ServiceNow / email / PagerDuty → team X's destination
```

No separate profile object exists in this shape. **The filter *is* the trigger.** Dynatrace notes that using a workflow trigger *"eliminates the need for adding and managing a separate alerting profile to filter incoming problems"*, and that the filter *"can use DQL matchers on the incoming problem record, which is much more flexible and powerful than alerting profiles."*

Status of the classic surfaces

Neither alerting profiles nor classic problem notifications are deprecated. Both are labeled **Dynatrace Classic**, and the docs recommend simple workflows for new setups. No end-of-life date has been published for either.

The forcing function is not deprecation — it is **the Management Zone itself**. A profile scoped by an MZ is only as durable as that MZ. Profiles scoped purely by severity rules and tags are unaffected by this migration and can be left alone.

> **Sources:** Upgrade guide — alerting and notifications (DT docs) — the "Management Zone filter: no longer supported" line. Event trigger (DT docs) — trigger filter surface and DQL-matcher scope. Alerting profiles (DT docs) — Dynatrace Classic status. Use segments in anomaly detection (DT docs) — the detector-scoping exception.

3. Inventory Before You Start

Alerting-profile configuration is not queryable via DQL — it lives in Settings. Export it via the Settings API (`builtin:alerting.profile`) or the UI, the same way MZ2POL-00 exports Management Zones.

Collect these five columns for every profile. Each one drives a decision later, and three of them surface capability regressions that are much cheaper to find now than at cutover.

Column	Why it matters	Drives
Management zone reference	Identifies which profiles this migration actually touches. Profiles without one need no work	Scope
Severity rules (level + tag filters)	Carries forward directly onto the workflow trigger	§5
`delayInMinutes` per severity rule	**No equivalent as of 07/2026** — re-check before cutover (§6.1). Every non-zero value needs a recorded disposition	§6
Event filters (predefined / custom title / description)	Becomes the DQL matcher on the trigger	§5

Column	Why it matters	Drives
Notification destinations consuming this profile	Four destination types have no native connector	§6

The three questions the inventory must answer

1. **How many profiles reference a Management Zone?** This is the real work list — often far smaller than the total profile count.
2. **How many use a non-zero `delayInMinutes`?** Every one is a potential noise regression at cutover unless deliberately handled — and together they define your *last* migration wave, not your first (§6.1).
3. **What destinations are in play?** Opsgenie, Trello, VictorOps, and xMatters need HTTP rebuilds.

> **Consolidate while you inventory.** Profiles are frequently near-duplicates that differ only by zone. The target is one workflow per **team/channel**, not one per profile — see §5. A hundred MZ-scoped profiles routinely collapse to a much smaller set of destinations.

4. Converting MZ Rules into Routing Dimensions

This is the actual work, and it is enrichment work, not alerting work.

The conversion table

Classify every MZ rule by whether its dimension is already carried as a **tag on the entity**. The right-hand column is your prerequisite backlog.

MZ rule shape	Tag-carried today?	What to do
Host tag equals `team:checkout`	✅ Yes	Direct mapping — use the tag on the trigger
Host group membership	⚠️ Derivable	Auto-tag from host group, then verify propagation

MZ rule shape	Tag-carried today?	What to do
Kubernetes namespace / cluster	⚠️ Derivable	Auto-tag from the namespace, then verify
`dt.security_context` already set	✅ Yes	Usable directly; also the visibility field (§7)
Service name **contains** `payment`	❌ Computed	**Must** become an auto-tag before routing can use it
Entity selector expression	❌ Computed	**Must** become an auto-tag
Process/host **name pattern**	❌ Computed	**Must** become an auto-tag

A trigger cannot evaluate *"service name contains payment"*. It can only ask whether the affected entity carries a tag. Every ❌ row is work that must complete — **and propagate** — before the corresponding workflow can be built.

Auditing tag coverage

Run these before building anything. Both are cheap: Smartscape queries scan **0 bytes**.

Replace `team` with whichever tag key carries your routing dimension.

```
// Routing-tag coverage on services.
// Live-verified 07/21/2026 — scans 0 bytes.
// On Smartscape nodes `tags` is a RECORD: use unquoted bracket access, not matchesValue().
smartscapeNodes "SERVICE"
| fieldsAdd team = tags[team]
| summarize total = count(), tagged = countIf(isNotNull(team))
| fieldsAdd coverage_pct = round(tagged * 100.0 / total, decimals: 1)
```

A `coverage_pct` below 100 is the size of your §4 backlog. Routing cannot be correct while it is below 100 — untagged entities produce problems that match no workflow and page nobody.

The same audit across two dimensions at once, on hosts:

```
// Multi-dimension routing-tag coverage on hosts.
// Live-verified 07/21/2026 — scans 0 bytes.
smartscapeNodes "HOST"
| fieldsAdd team = tags[team], env = tags[environment]
| summarize hosts = count(),
            with_team = countIf(isNotNull(team)),
            with_env = countIf(isNotNull(env))
| fieldsAdd team_pct = round(with_team * 100.0 / hosts, decimals: 1),
            env_pct = round(with_env * 100.0 / hosts, decimals: 1)
```

> ⚠️ **`tags` behaves differently on Smartscape nodes than on classic entities.** On a Smartscape node it is a **record** — access it with unquoted bracket syntax, `tags[team]`. On a classic entity it is an **array of `"key:value"` strings**, where OneAgent host tags additionally carry an `[Environment]` prefix. Using `matchesValue(tags, "team:checkout")` against `smartscapeNodes` does not merely underperform — `verify_dql` reports that the query *"will always return an empty result as the condition can't be true."* MZ2POL-05 §4.5 documents this trap in full.

Propagation is not instant

Creating an auto-tag rule does not retroactively tag existing entities at once. Re-run the coverage queries until they reach 100% before building the workflow that depends on them — a trigger filtering on a tag that has only half propagated will silently under-route.

5. Building the Replacement Workflow

The full trigger filter surface

These are all the controls a problem trigger offers:

Setting	What it does
Problem state	Active only, or active + closed
Categories	Davis problem categories
Severity	Minimum severity threshold

Setting	What it does
Affected entities — tags	Three modes: include all / all defined tags / any defined tag. Evaluated over the union of `affected_entity_ids` and `smartscape.affected_entities`
Additional custom filter query	A DQL **matcher** on the problem record — a *subset* of DQL: no aggregation, no querying across a set of events
Updates	Re-trigger when selected fields change

No management-zone filter. No segment filter. No entity-selector filter.

Granularity: one simple workflow per destination

Approach	Object count	Verdict
One workflow per Management Zone	= MZ count	❌ Unmanageable; most would be near-duplicates
One simple workflow per team/channel	= destination count	✅ Recommended
One workflow branching to all teams	1	❌ Single point of failure; multi-step billing; hard to review

A **simple** workflow — one trigger, one notification action — avoids the workflow-hour consumption that multi-step workflows incur. Adding an HTTP action or branching logic can move it into the billed category. ALERT-03 covers that boundary; re-verify the current billing rule against your contract before committing to a fleet size.

Do not route on the management-zone payload field

`event()["management_zones"]` still appears in the problem payload, so an expression reading it *looks* like it works.

After the zones are deleted it resolves to an **empty array**. Every condition built on it goes false, and the workflow stops notifying **without raising an error**.

Silent alert loss is the worst failure mode available in this migration, and this is the easiest way to cause it. WFLOW-04 §4 documents the pattern as legacy for exactly this reason.

Manage as code

At any meaningful profile count, hand-building in the UI produces drift within weeks. Template the first workflow, then generate the rest — see **AUTOM-04** (Terraform) or **AUTOM-03** (Monaco).

6. Capability Regressions

Two things get worse. Both are cheaper to plan for than to discover.

6.1 Duration-based suppression has no successor today

An alerting profile can delay notification until a problem has been open longer than *N* minutes (`delayInMinutes`). Teams use it to suppress transient blips.

As of 07/2026 the workflow model has no equivalent. The upgrade guide states there is *"currently no alternative"* for delivering problems that have been active longer than *X* minutes.

> **Read that "currently" as load-bearing.** It is the upgrade guide's own wording, and it marks this as a stated gap rather than a settled design decision. **Re-check the upgrade guide before you commit a cutover wave** — of everything in this notebook, this is the claim most likely to have changed since it was written, and a reader acting on a stale copy would accept a regression they may no longer have to.

This is the one **hard capability regression** in the migration today. It cannot be designed around inside the alerting layer, and it is worth naming to on-call teams *before* cutover — every other part of this migration is neutral-to-better for them, and this one is strictly worse. Discovered afterwards, it reads as *"the migration made my pager noisier."*

Per-profile options:

Option	When it fits	Trade-off
Defer the profile to a later wave	The delay is load-bearing and the team will not accept the regression	Costs only time — and this is the one cohort with a concrete reason to wait

Option	When it fits	Trade-off
Accept the noise	Delay was short; the signal is genuinely actionable	Slight increase in short-lived pages
Replace with a Davis anomaly detector	The signal was noisy because a static threshold was wrong for it	Changes detection semantics, not just routing; needs tuning
Express as an SLO burn-rate alert	The concern was sustained degradation, not a threshold crossing	Best conceptual fit; requires an SLO to exist (**SLO-04**)
Retire the alert	The delay was masking an alert nobody acts on	Free noise reduction — check usage first

Sequence delay-dependent profiles last. They are the only profiles whose behavior you cannot reproduce, so they are the only ones with a reason to wait. Profiles with `delayInMinutes: 0` — usually the large majority — carry no such constraint and should not be held back with them.

> **A long `delayInMinutes` is usually evidence the alert was the wrong *shape*, not merely delayed.** A profile suppressing 30 minutes of a firing condition is describing a burn-rate concern. Route those to SLO burn-rate alerts where an SLO exists, and to Davis where one does not.

6.2 Four destinations have no native connector

Documented connector mapping:

Classic notification	Workflow equivalent
Ansible	RedHat Ansible connector
Custom integration	HTTP Request action
Email	Microsoft 365 / Email connector
Jira	Jira connector
PagerDuty	PagerDuty connector
ServiceNow	ServiceNow connector

Classic notification	Workflow equivalent
Slack	Slack connector

Opsgenie, Trello, VictorOps, and xMatters are currently not supported as native workflow connectors. Each becomes an HTTP-action rebuild: reconstruct the payload against the destination's current API, store credentials in the vault, and accept that the workflow may now count as multi-step for billing.

Rebuild the payload against the destination's current API contract — do not port the old webhook body verbatim.

A path that removes work rather than adding it

The **Problems app personal email subscription** lets an individual subscribe to a filter in the Problems app with no workflow and no configuration permission. It is triggered within OpenPipeline, so only simple filters apply.

It is not team routing — but it does cover a real category: people who were added to an MZ-scoped email profile purely for awareness. Moving them to a personal subscription removes them from the workflow fleet entirely.

> **Unverified:** whether the personal email subscription honors an active *segment* as its filter. The docs say only "simple filters." Confirm in your tenant before relying on it.

7. Problem Visibility Is a Separate Axis

If a Management Zone was doing double duty — restricting which problems a team could **see**, not just which ones paged them — then replacing the routing does **not** replace the visibility.

	Routing	Visibility
Mechanism	Workflow trigger filter	IAM record-level policy
Field	Entity tags + DQL matcher	`dt.security_context`
Failure if wrong	Wrong team paged	Wrong team **sees data they should not**

The constraint that decides the design

When multiple events aggregate into a problem, the values of a given field are combined into an **array**. Because of how Grail record-level permission filters currently work, **only `dt.security_context` supports filtering array values.**

> A problem-visibility model built on team tags, a custom attribute, or any field other than `dt.security_context` **will not work**. This is not a performance note or a preference — the filter will not behave correctly once events aggregate.

Dynatrace propagates `dt.security_context` from an event's `dt.source_entity` onto alerting events specifically so IAM policy rules can control alert and problem visibility at the record level (SaaS 1.333).

This is the same field the access-control half of the migration uses, which is convenient: MZ2POL-02 and MZ2POL-04 already establish it. If those notebooks' work is done, visibility is largely already handled — verify rather than rebuild.

> **Sources:** Problems app (DT docs) — the array-filtering constraint and the personal email subscription. SaaS 1.333 release notes (DT docs) — `dt.security_context` propagation onto alerting events.

8. Cutover Sequencing and the Deletion Test

The order

```
1. Build workflows            — with tags verified at 100% coverage (§4)
2. Run in parallel            — classic profiles AND new workflows both active
3. Prove alert parity         — per-team volume matches, for an agreed window
4. DISABLE classic profiles   — not delete; disable, so rollback is one click
5. Soak                       — agreed quiet period
6. Delete Management Zones    — SEPARATE change window, last
```

Steps 4 and 6 must not share a change window. After a combined change there is no way to tell which half caused a regression.

The unresolved question

> ⚠️ **Undocumented behavior — test it, do not assume it.**
>
> **What happens to an alerting profile whose Management Zone reference is deleted?**
>
> - **Fails open** — the profile matches everything. Across a large profile estate, a page storm.
> - **Fails closed** — the profile matches nothing. Silent alert loss, which is worse, because nothing announces it.
>
> This is not documented, and it could not be verified while writing this notebook. Establish it yourself before deleting any production Management Zone.

The test procedure

Run in a **non-prod tenant**.

Step	Action	Record
1	Create a throwaway MZ scoped to a small, controllable entity set	MZ name, scope
2	Create an alerting profile scoped to that MZ, routing to a test destination	Profile config
3	Trigger a problem on an **in-scope** entity; confirm the notification fires	Positive baseline
4	Trigger a problem on an **out-of-scope** entity; confirm it does **not** fire	Negative baseline
5	Delete the Management Zone. Change nothing else	Deletion timestamp
6	Re-run step 3	Fires / does not fire
7	Re-run step 4	**This is the answer.** Fires ⇒ fail-open. Does not fire ⇒ fail-closed

Step 4 is the one people skip, and it is the one that matters — without a negative baseline, step 7 proves nothing.

What the answer changes

Result	Consequence
Fail-open	MZs must be deleted only *after* profiles are deleted, not merely disabled — otherwise every dangling profile pages everyone
Fail-closed	Safer, but demands parity proof *before* deletion, since failures will be silent

Retire Management Zones last

Classic apps still depend on Management Zones, and so do any alerting profiles you chose to keep. MZ2POL-05 §8 covers coexistence; the ordering rule is the same here — the MZ is the last thing to go.

9. Validating the Migration

Alert parity is the acceptance test

The only meaningful proof is that each team receives the same pages after cutover as before. Compare per-team alert volume across the parallel-run window.

> **This is the one thing the platform team cannot validate alone.** Only the team currently receiving an MZ-scoped page can confirm they still receive the equivalent page. Budget for their time explicitly — this is the most common source of cutover slippage.

Problem distribution by security context

Establishes the baseline distribution to compare against after cutover:

```
// Problem-event volume by security context over the parallel-run window.
// Executed 07/31/2026 over 7 days; results below the cell.
// Counts problem-EVENT records, not distinct problems - see the note that
follows.
fetch events, from: -7d
| filter event.kind == "DAVIS_PROBLEM"
| expand dt.security_context
| summarize problems = count(), by:{dt.security_context}
| sort problems desc
| limit 20
```

`dt.security_context` is an **array** on problem records once events aggregate — hence the `expand`. This is the same array behavior that constrains the visibility model in §7.

Security-context coverage on problems

A problem with no security context cannot be filtered for visibility and may not route as expected:

Executed against a validation tenant over 7 days on 07/31/2026, the top rows were:

`dt.security_context`	problem-event records
(null)	2,319,333
`ENV | Prod`	147,055
`Elevate | Prod | Monitoring Team`	77,689
`Elevate | Prod`	73,533
`Prod All (old)`	73,522

Read the unit before you read the numbers. `fetch events | filter event.kind == "DAVIS_PROBLEM"` returns the raw problem-event stream — many records per problem, one for each state update over its life — so these are *records*, not distinct problems. On the same tenant and window, `fetch dt.davis.problems` returned 3,318 rows against 2,480,695 problem-event records: an over-count of roughly 750×. That is fine for a *relative* comparison across contexts, which is what this baseline is for, and badly wrong if you quote it as a problem count to anyone. ALERT-99 §3 sets out all three Davis surfaces and which question each answers.

Note also the shape of that top row: the largest bucket by far is the one with **no security context at all**.

```
// Security-context coverage across the problem-event stream.
// Executed 07/31/2026 over 7 days: total 2,480,695 / with_context 161,361 /
6.5%.
fetch events, from: -7d
| filter event.kind == "DAVIS_PROBLEM"
| summarize total = count(), with_context =
countIf(isNotNull(dt.security_context))
| fieldsAdd coverage_pct = round(with_context * 100.0 / total, decimals: 1)
```

On the validation tenant, 07/31/2026, over 7 days:

`total`	`with_context`	`coverage_pct`
2,480,695	161,361	**6.5**

93.5% of the problem-event stream carries no security context at all. That number is the argument of this notebook in a single figure.

The migration story people reach for is that MZ-scoped alerting will be rebuilt on a field like `dt.security_context` — swap one scoping dimension for another and the routing follows. This measurement is what that plan collides with. A field populated on 6.5% of records is not a scoping dimension yet; it is an enrichment project that has not happened. Build routing on it today and 93.5% of the stream falls through to whatever the default is — which, for alerting, means either everyone is paged or nobody is.

That is why §4 puts enrichment first and treats it as the prerequisite rather than a parallel workstream. **Run this query before you commit to a cutover date, not after.** A tenant at 6.5% has months of tagging and propagation work in front of it; a tenant already near 100% can move quickly. The number is cheap to obtain and it is the difference between a schedule and a guess.

A coverage figure this low is also worth a second look before you treat it as pure bad news: check whether the contexts that *are* populated cover the entities that actually page people. Coverage weighted by alert-carrying entities is the number that matters operationally, and it is often better than the raw record-level figure — the null bucket is inflated by high-frequency events on unenriched infrastructure.

What to check before declaring done

Check	Pass condition
Routing-tag coverage (§4)	100% on every entity type in scope

Check	Pass condition
Workflow per destination exists	Every inventoried destination has a live workflow
Alert parity per team	Within agreed tolerance across the parallel window, **confirmed by the receiving team**
No workflow reads `management_zones`	Zero matches across all workflow definitions
`delayInMinutes` dispositions	Every non-zero value has a recorded decision
Security-context coverage	100%, or every gap explained
Deletion behavior established	§8 test run and result recorded

10. Checklist

Inventory

- [ ] Export alerting profiles (`builtin:alerting.profile`) — Settings API or UI
- [ ] Identify which profiles reference a Management Zone (the real work list)
- [ ] Record `delayInMinutes` per severity rule
- [ ] Record every notification destination and its type
- [ ] Identify near-duplicate profiles that collapse to one destination

Enrichment (prerequisite)

- [ ] Classify every MZ rule as tag-carried / derivable / computed (§4)
- [ ] Create auto-tags for every computed dimension
- [ ] Run the coverage queries until they reach 100%
- [ ] Confirm propagation has completed, not just that rules exist

Build

- [ ] One simple workflow per team/channel — not per Management Zone
- [ ] Trigger filters on affected-entity tags, refined by DQL matcher if needed
- [ ] Carry severity rules across from the old profile
- [ ] Rebuild the four unsupported destinations as HTTP actions
- [ ] Template and manage as code (AUTOM-03 / AUTOM-04)
- [ ] Verify no workflow reads `event()["management_zones"]`

Regressions

- [ ] Every non-zero `delayInMinutes` has a recorded disposition
- [ ] On-call teams told about the duration-suppression loss *before* cutover
- [ ] Move awareness-only recipients to Problems-app personal subscriptions

Visibility

- [ ] Visibility model uses `dt.security_context` and nothing else
- [ ] IAM record-level policies in place and tested

Cutover

- [ ] Deletion-behavior test run in non-prod; result recorded (§8)
- [ ] Parallel run complete; alert parity confirmed **by the receiving teams**
- [ ] Classic profiles **disabled**, not deleted
- [ ] Soak period elapsed
- [ ] Management Zones deleted in a **separate** change window, last

---

Summary

1. **Alerting is the third job a Management Zone does**, and it migrates to problem-triggered workflows — not to Segments. Segments scope queries and anomaly detectors; they never scope triggers, notifications, or visibility.
2. **The real work is enrichment.** Triggers match tags carried by entities; MZ rules are computed conditions. Every computed dimension must become an auto-tag, and must propagate, before its workflow can exist.
3. **Two capability regressions:** duration-based suppression has no equivalent, and four notification destinations have no native connector.
4. **Visibility is a separate axis** and works only on `dt.security_context`, because it is the only field that filters correctly once events aggregate into a problem.
5. **The deletion failure mode is undocumented.** Test it in non-prod before touching production, and never delete zones and profiles in the same change window.

Next Steps

- **MZ2POL-06** — migration execution, phasing, and rollback for the migration as a whole
- **MZ2POL-07** — validation and troubleshooting
- **MZ2POL-99** — the consolidated best-practice checklist across the series

Additional Resources

- Upgrade guide — alerting and notifications (DT docs) — the authoritative old→new mapping; the Management Zone filter statement, the connector table, and the duration-filter gap
- Event trigger (DT docs) — problem-trigger filter surface and DQL-matcher scope
- Alerting and notifications (DT docs) — the current Gen3 hub
- Alerting profiles (DT docs) — Dynatrace Classic status
- Problem notifications (DT docs) — classic notification status
- Problems app (DT docs) — array-filtering constraint; personal email subscription
- Use segments in anomaly detection (DT docs) — the one documented segment/alerting intersection
- SaaS 1.333 release notes (DT docs) — `dt.security_context` propagation onto alerting events

---